

1. Capa

FATEC – Faculdade de Tecnologia

Curso: Desenvolvimento de Software Multiplataforma

PROJETO 01 – DESENVOLVIMENTO WEB III

Aplicação Web para Apresentação Pessoal

Integrantes:

Maciel dos Santos

Integrante 2 – A definir

Professor: Vinícius Heltai Pacheco

2026

2. Identificação dos integrantes

Maciel dos Santos

Aluno do curso de Desenvolvimento de Software Multiplataforma da FATEC.

Segundo integrante:

A definir.

3. Objetivo do projeto

O Projeto 01 da disciplina de Desenvolvimento Web III tem como objetivo desenvolver uma aplicação web para apresentação pessoal dos integrantes da equipe, utilizando os conceitos estudados durante as aulas.

A aplicação foi desenvolvida utilizando Node.js sem framework, empregando os módulos nativos `http`, `url` e `fs`. O projeto aplica conceitos de criação de servidor HTTP, funções callback, rotas (endpoints), leitura de arquivos e disponibilização de páginas HTML e documentos PDF através do servidor web.

4. Estrutura das rotas

Rota	Função	Arquivo
/	Página inicial	index.html
/maciel	Página pessoal	maciel/index.html
/maciel/sobre	Apresentação pessoal	maciel/sobre.html
/maciel/curriculo	Currículo	maciel/curriculo_Maciel.pdf
/integrante2	Página pessoal	integrante2/index.html
/integrante2/sobre	Apresentação pessoal	integrante2/sobre.html
/integrante2/curriculo	Currículo	integrante2/curriculo_integrante2.pdf
/projeto	Documentação	projeto/documentacao.html
/css/estilo.css	Folha de estilos	css/estilo.css
Outras rotas	Página de erro	erro404.html

5. Funcionamento geral

Fluxo:

Usuário acessa uma URL



Servidor Node.js recebe a requisição



url.parse(request.url)



Identificação da rota



if / else if



fs.readFile()



Arquivo HTML, CSS ou PDF



response.end()



Conteúdo exibido no navegador

6. Explicação do funcionamento da aplicação

A aplicação foi desenvolvida utilizando **Node.js sem framework**, seguindo a evolução dos exemplos `app01.js`, `app02.js`, `app03.js` e `app04.js` apresentados em aula.

O funcionamento começa pelo carregamento de três módulos nativos:

```
const http = require('http');
const url = require('url');
const fs = require('fs');
```

Cada módulo possui uma responsabilidade.

Módulo	Responsabilidade
<code>http</code>	Criar e executar o servidor web
<code>url</code>	Interpretar a URL e identificar a rota
<code>fs</code>	Ler os arquivos existentes no projeto

6.1 Módulo `http`

```
const http = require('http');
```

O módulo `http` permite criar o servidor web.

No final do programa temos:

```
var server = http.createServer(callback);
```

Aqui o Node cria o servidor e informa que a função:

`callback`

será responsável por processar as requisições recebidas.

Depois:

```
server.listen(3000);
```

faz o servidor ficar aguardando conexões na porta `3000`.

Por isso acessamos:

```
http://localhost:3000/
```

6.2 Função callback

```
var callback = function(request, response) {
```

Temos dois parâmetros importantes:

`request`

representa a **requisição feita pelo navegador**.

E:

`response`

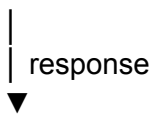
representa a **resposta que o servidor enviará ao navegador**.

Podemos pensar assim:

NAVEGADOR



SERVIDOR NODE.JS



NAVEGADOR

Esse conceito veio principalmente do `app02.js`.

6.3 Identificação da URL

Dentro do callback temos:

```
var parts = url.parse(request.url);
```

Essa linha utiliza o módulo `url` para interpretar o endereço solicitado.

Por exemplo, quando o usuário acessa:

`http://localhost:3000/maciел/sobre`

a aplicação consegue verificar:

`parts.path`

e identificar:

/maciel/sobre

Esse conceito veio do `app03.js`.

6.4 Verificação das rotas

Depois utilizamos:

```
if  
else if  
else
```

para descobrir qual rota foi solicitada.

Por exemplo:

```
else if (parts.path == "/maciel/sobre") {  
  
    response.writeHead(200, {"Content-type": "text/html"});  
    readFile(response, "maciel/sobre.html");  
  
}
```

Aqui estamos dizendo:

SE o usuário acessar
/maciel/sobre

ENTÃO

abra
maciel/sobre.html

Essa distinção é importante:

/maciel/sobre

é a **rota**.

Enquanto:

maciel/sobre.html

é o **arquivo físico**.

6.5 `response.writeHead()`

Temos comandos como:

```
response.writeHead(200, {"Content-type": "text/html"});
```

O primeiro valor:

200

é o código HTTP.

200 significa que a requisição foi processada com sucesso.

O segundo informa o tipo de conteúdo:

text/html

No nosso projeto utilizamos diferentes tipos:

text/html

application/pdf

text/css

Portanto:

```
{"Content-type": "text/html"}
```

para páginas HTML.

```
{"Content-type": "application/pdf"}
```

para os currículos e, posteriormente, documentação.

```
{"Content-type": "text/css"}
```

para a folha de estilos.

6.6 Função `readFile()`

Essa é uma das partes principais do projeto:

```
function readFile(response, file) {  
  fs.readFile(file, function(err, data) {  
    if (err) {
```

```

        console.log("ERRO AO ABRIR:", file);
        console.log(err);

        response.end("Erro ao abrir o arquivo: " + file);
    }
    else {
        console.log("Arquivo aberto com sucesso:", file);

        response.end(data);
    }

});

}

```

Ela evita precisar escrever `fs.readFile()` novamente em cada rota.

Por exemplo:

```
readFile(response, "index.html");
```

ou:

```
readFile(response, "maciel/sobre.html");
```

ou:

```
readFile(response, "maciel/curriculo_Maciel.pdf");
```

O parâmetro `file` recebe o caminho do arquivo.

Depois:

```
fs.readFile(file, function(err, data) {
```

tenta ler esse arquivo.

Se ocorrer algum problema:

```
if (err)
```

o erro é mostrado no terminal.

Se der certo:

```
else
```

o conteúdo é enviado:

```
response.end(data);
```

Esse conceito é a principal evolução apresentada no `app04.js`.

6.7 Erro 404

No final das verificações temos:

```
else {  
  
    response.writeHead(404, {"Content-type": "text/html"});  
    readFile(response, "erro404.html");  
  
}
```

Se nenhuma rota anterior corresponder ao endereço solicitado, o servidor responde com:

404

que representa **recurso/página não encontrado**.

E abre:

erro404.html

Por exemplo:

localhost:3000/qualquercoisa

↓

nenhuma rota encontrada

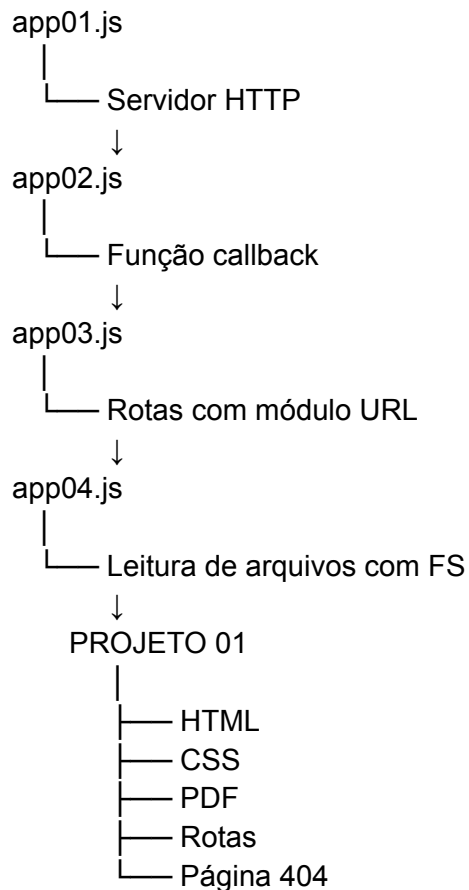
↓

404

↓

erro404.html

7. Relação com os exercícios da aula



8. Justificativa das decisões adotadas

8.1 Uso do Node.js sem framework

O projeto foi desenvolvido utilizando **Node.js sem framework**, seguindo a metodologia apresentada nas aulas de Desenvolvimento Web III.

Essa escolha permite aplicar diretamente os conceitos estudados nos arquivos `app01.js`, `app02.js`, `app03.js` e `app04.js`, principalmente a criação de servidor HTTP, utilização de callback, definição de rotas e leitura de arquivos.

8.2 Organização das rotas

As rotas foram organizadas de forma hierárquica para facilitar a navegação e a identificação dos conteúdos.

Para o primeiro integrante, por exemplo:

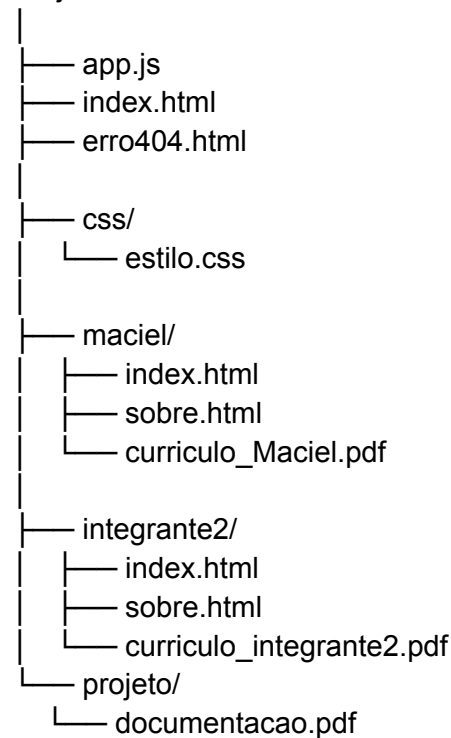
```
/maciel  
/maciel/sobre  
/maciel/curriculo
```

Dessa forma, todas as informações relacionadas ao aluno Maciel ficam agrupadas sob a rota `/maciel`.

8.3 Organização dos arquivos

Os arquivos foram separados em diretórios de acordo com suas funções:

Projeto01-DW3/



A pasta `maciel` concentra os arquivos relacionados ao primeiro integrante, enquanto `css` armazena a folha de estilos e `projeto` é destinada à documentação.

Essa separação facilita a manutenção e a localização dos arquivos.

8.4 Utilização de um CSS compartilhado

Foi utilizado um único arquivo:

`css/estilo.css`

para definir a aparência das diferentes páginas HTML.

As páginas fazem a referência:

```
<link rel="stylesheet" href="/css/estilo.css">
```

Isso evita repetir as mesmas regras CSS em vários arquivos e mantém uma identidade visual consistente.

8.5 Currículo em PDF

O currículo é disponibilizado por meio da rota:

/maciel/curriculo

O servidor responde utilizando:

```
response.writeHead(200, {"Content-type": "application/pdf"});
```

e entrega:

```
readFile(response, "maciel/curriculo_Maciel.pdf");
```

Essa decisão permite que o usuário acesse o currículo diretamente por uma rota da aplicação, conforme solicitado nos requisitos.

8.6 Página de erro 404

Também foi criada uma página específica:

erro404.html

Quando o usuário solicita uma rota inexistente, o servidor retorna o código HTTP:

404

e apresenta uma página que permite retornar à página inicial.

9. Considerações finais

O desenvolvimento do Projeto 01 permitiu aplicar de forma prática os conteúdos estudados nas aulas de Desenvolvimento Web III. A partir dos exemplos `app01.js`, `app02.js`, `app03.js` e `app04.js`, foi possível compreender a evolução desde a criação de um servidor HTTP básico até uma aplicação capaz de trabalhar com diferentes rotas e disponibilizar arquivos HTML, CSS e PDF.

O projeto também contribuiu para o aprendizado sobre organização de arquivos, navegação entre páginas, códigos de resposta HTTP e utilização dos módulos nativos `http`, `url` e `fs` do Node.js. A aplicação foi estruturada para permitir a inclusão e atualização das informações dos integrantes da equipe.